

WEEK 5: ETHICAL HACKING & EXPLOITING VULNERABILITIES

Complete Security Assessment & Hardening Report

Project Name: User Management System - Security Hardening

Intern Name: Muhammad Talha

Week: 5 (Ethical Hacking & Exploiting Vulnerabilities)

Date: 06 July 2026

Deadline: 14 July 2026

EXECUTIVE SUMMARY

This report documents the comprehensive ethical hacking assessment conducted on the DVWA (Damn Vulnerable Web Application) testing environment during Week 5 of the Cybersecurity Internship. The primary objectives were to identify, exploit, and remediate critical vulnerabilities including SQL Injection and Cross-Site Request Forgery (CSRF). All testing was performed in an isolated local environment using industry-standard penetration testing tools.

Key Achievements:

-  Successfully identified and exploited SQL Injection vulnerability
-  Extracted and cracked 5 user passwords from database
-  Identified and exploited CSRF vulnerability
-  Implemented secure coding fixes (Prepared Statements + CSRF Tokens)
-  Hardened server configuration (Security Headers, PHP settings)
-  Comprehensive reconnaissance performed (Nmap, WhatWeb, Nikto)

1. INTRODUCTION

1.1 Objective

The primary objective of Week 5 was to:

- Think like an attacker and identify vulnerabilities
- Exploit vulnerabilities in a controlled test environment
- Apply security fixes to prevent real-world attacks
- Document findings and remediation steps

1.2 Scope

- **Target:** DVWA (Damn Vulnerable Web Application)
- **Environment:** Localhost (192.168.229.1)
- **Testing Tools:** Nmap, WhatWeb, Nikto, SQLMap, Burp Suite
- **Vulnerabilities Tested:** SQL Injection, CSRF, Information Disclosure

2. RECONNAISSANCE (Information Gathering)

2.1 Nmap Scan - Port Scanning

Command:

```
nmap -sV 192.168.229.1
```

Results:

PORT	STATE SERVICE	VERSION
80/tcp	open http	Apache httpd 2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
135/tcp	open msrpc	Microsoft Windows RPC
139/tcp	open netbios-ssn	Microsoft Windows netbios-ssn
443/tcp	open ssl/http	Apache httpd 2.4.58
445/tcp	open	microsoft-ds?
3306/tcp	open mysql	MariaDB 10.3.23

Findings:

Port	Service	Version	Risk
80	HTTP	Apache 2.4.58	⚠ Outdated
443	HTTPS	Apache 2.4.58	⚠ Outdated
3306	MySQL	MariaDB 10.3.23	⚠ Outdated

Recommendations:

- Update Apache to latest version (2.4.66+)

- Update MariaDB to latest version
- Restrict MySQL access to localhost only

2.2 WhatWeb Scan - Technology Detection

Command:

```
whatweb http://192.168.229.1/DVWA/
```

Results

http://192.168.229.1/DVWA/ [200 OK]

Apache[2.4.58], Cookies[PHPSESSID,security], HTTPServer[Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12]

PHP[8.2.12], X-Frame-Options[DENY], Strict-Transport-Security[max-age=31536000; includeSubDomains]

Content-Security-Policy[default-src 'self'], X-Content-Type-Options[nosniff], Referrer-Policy[no-referrer]

Findings:

-  Security Headers implemented (Week 4 work)
-  PHP version exposed (8.2.12)
-  Apache version exposed (2.4.58)

Recommendations:

- Hide PHP version (expose_php = Off)
- Hide Apache version (ServerTokens Prod)

2.3 Nikto Scan - Vulnerability Scanner

Command:

```
nikto -h http://192.168.229.1/DVWA/login.php
```

Before Fixes Findings:

Finding	Risk	Status
PHP version exposed (X-Powered-By)	Low	✗ Vulnerable
Cookie without HttpOnly flag	Medium	✗ Vulnerable
TRACE method enabled	High	✗ Vulnerable
Missing Permissions-Policy header	Medium	✗ Vulnerable
OpenSSL outdated (3.1.3)	Medium	⚠ Warning
Apache outdated (2.4.58)	Medium	⚠ Warning

3. SECURITY HARDENING - CONFIGURATION FIXES

3.1 PHP Configuration (php.ini)

File: C:\xampp\php\php.ini

Changes Applied:

Before: expose_php = On

expose_php = Off

Before: session.cookie_httponly =

session.cookie_httponly = 1

Before: session.cookie_samesite =

session.cookie_samesite = Strict

Impact:

- ☒ PHP version hidden from headers
- ☒ HttpOnly flag added to cookies
- ☒ SameSite=Strict added for CSRF protection

3.2 Apache Configuration (httpd.conf)

File: C:\xampp\apache\conf\httpd.conf

Changes Applied:

Disable TRACE method

TraceEnable Off

Hide server version

ServerTokens Prod

ServerSignature Off

Add Permissions-Policy header

Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"

Impact:

- ☒ TRACE method disabled (XST prevention)
- ☒ Apache version hidden
- ☒ Permissions-Policy header added

3.3 Nikto Scan - After Fixes

Results:

+ Server: Apache/2.4.58

- ☒ No X-Powered-By header
- ☒ Cookie PHPSESSID has HttpOnly flag
- ☒ Permissions-Policy header present
- ☒ TRACE method disabled

Remaining Warnings:

-  Apache 2.4.58 outdated (update recommended)
-  OpenSSL 3.1.3 outdated (update recommended)

4. SQL INJECTION TESTING

4.1 Vulnerability Discovery

Target: <http://192.168.229.1/DVWA/vulnerabilities/sqli/>

Test Method: SQLMap Automated Tool

Command:

```
sqlmap -u "http://192.168.229.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --  
cookie="PHPSESSID=abc123; security=low" --batch
```

Findings

Parameter: id (GET)

Type: boolean-based blind

Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)

Payload: id=1' OR NOT 8511=8511#&Submit=Submit

Type: error-based

Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)

Payload: id=1' AND EXTRACTVALUE(3209,CONCAT(0x5c,0x71707a7071,(SELECT (ELT(3209=3209,1))),0x7178766b71))-- ofCH&Submit=Submit

Type: time-based blind

Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)

Payload: id=1' AND (SELECT 7850 FROM (SELECT(SLEEP(5)))CgNi)-- VCgm&Submit=Submit

Type: UNION query

Title: MySQL UNION query (NULL) - 2 columns

Payload: id=1' UNION ALL SELECT NULL,CONCAT(...)#&Submit=Submit

Impact: Complete database access possible

4.2 Data Extraction

Databases Found:

Database	Purpose
dvwa	Main application database (TARGET)
information_schema	MySQL metadata
mysql	User accounts
performance_schema	Performance data
phpmyadmin	phpMyAdmin config
test	Empty test DB

Tables in dvwa Database:

[2 tables]

+-----+

| guestbook |

| users |

+-----+

Users Table Data:

ID	User	Password (Cracked)	Role
1	admin	password	Admin
2	gordonb	abc123	User
3	1337	charley	User
4	pablo	letmein	User
5	smithy	password	User

Password Cracking:

```
sqlmap -u "http://192.168.229.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --
cookie="PHPSESSID=abc123; security=low" -D dvwa -T users --dump --batch
```

Cracked Passwords:

```
[10:19:16] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
```

```
[10:19:19] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
```

```
[10:19:28] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
```

```
[10:19:30] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
```

4.3 SQL Injection Fix - Prepared Statements

File: C:\xampp\htdocs\DVWA\vulnerabilities\sqli\source\low.php

Vulnerable Code:

```
<?php
```

```
if( isset( $_GET[ 'Submit' ] ) ) {
```

```
    $id = $_GET[ 'id' ];
```

```
    $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

```
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query );
```



```
}
```

```
?>
```

Fixed Code:

```
<?php
```

```
if( isset( $_GET[ 'Submit' ] ) ) {
```

```
    $id = $_GET[ 'id' ];
```

```
    //  Prepared Statement
```

```
    $stmt = mysqli_prepare($GLOBALS["__mysqli_ston"], "SELECT first_name, last_name  
FROM users WHERE user_id = ?");
```

```
    mysqli_stmt_bind_param($stmt, "i", $id);
```

```
    mysqli_stmt_execute($stmt);
```

```
    $result = mysqli_stmt_get_result($stmt);
```

```
}
```

```
?>
```

Verification: SQLMap retest failed (injection blocked) 

5. CSRF TESTING

5.1 Vulnerability Discovery

Target: <http://192.168.229.1/DVWA/vulnerabilities/csrf/>

Test Method: Burp Suite

Captured Request:

POST /DVWA/vulnerabilities/csrf/ HTTP/1.1

Host: 192.168.229.1

Cookie: PHPSESSID=abc123; security=low

password_new=hacked&password_conf=hacked&Change=Change

Issue: No CSRF token validation!

Attack Scenario:

1. Attacker creates malicious website
2. User visits website while logged into DVWA
3. Website automatically submits password change request
4. User's password is changed without their knowledge

5.2 CSRF Fix - Token Implementation

File: C:\xampp\htdocs\DVWA\vulnerabilities\csrf\source\low.php

Vulnerable Code:

```
<?php

if( isset( $_GET[ 'Change' ] ) ) {

    $pass_new = $_GET[ 'password_new' ];

    $query = "UPDATE users SET password = '$pass_new' WHERE user = 'admin'";

}

?>
```

Fixed Code:

```
<?php

session_start();

//  Generate CSRF token
```

```

if (!isset($_SESSION['csrf_token'])) {

    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));

}

if( isset( $_GET[ 'Change' ] ) ) {

    // ✅ Validate CSRF token

    $user_token = $_GET[ 'user_token' ] ?? '';

    if ($user_token !== $_SESSION['csrf_token']) {

        die("❌ CSRF token validation failed!");

    }

    $pass_new = $_GET[ 'password_new' ];

    // ✅ Hash password

    $hashed_password = password_hash($pass_new, PASSWORD_DEFAULT);

    $query = "UPDATE users SET password = '$hashed_password' WHERE user = 'admin'";

}

?>

```

Frontend Token Addition:

```

<form action="#" method="GET">

    New Password: <input type="password" name="password_new"><br>

    Confirm Password: <input type="password" name="password_conf"><br>

    <!-- ✅ CSRF Token -->

    <input type="hidden" name="user_token" value="<?php echo $_SESSION['csrf_token']; ?>">

    <input type="submit" name="Change" value="Change">

</form>

```

Verification: Burp Suite retest with token removed → Error message 

6. VULNERABILITY SUMMARY

6.1 Vulnerabilities Found & Fixed

Vulnerability	Severity	Status	Fix Applied
SQL Injection	 Critical	 Fixed	Prepared Statements
CSRF	 High	 Fixed	Token Validation
TRACE Method Enabled	 Medium	 Fixed	TraceEnable Off
PHP Version Exposed	 Low	 Fixed	expose_php = Off
Cookie Without HttpOnly	 Medium	 Fixed	session.cookie_http only = 1
Missing Permissions-Policy	 Medium	 Fixed	Header added
Apache Outdated	 Medium	 Pending	Update recommended
OpenSSL Outdated	 Medium	 Pending	Update recommended

6.2 Risk Assessment

Before Fixes:

Critical: 2 vulnerabilities (SQLi, CSRF)

High: 0 vulnerabilities

Medium: 4 vulnerabilities

Low: 1 vulnerability

After Fixes:

Critical: 0 vulnerabilities 

High: 0 vulnerabilities 

Medium: 2 vulnerabilities (Outdated software)

Low: 0 vulnerabilities 

7. TOOLS & TECHNOLOGIES

7.1 Testing Environment

- OS: Windows 11 + Kali Linux (VirtualBox)
- Web Server: XAMPP (Apache 2.4.58)
- Database: MySQL/MariaDB 10.3.23
- Target: DVWA

7.2 Tools Used

Tool	Purpose
Nmap	Port scanning, service detection
WhatWeb	Technology stack detection
Nikto	Web vulnerability scanning
SQLMap	SQL injection exploitation
Burp Suite	CSRF testing, request manipulation
GParted	Disk management
XAMPP	Local web server
Chrome DevTools	Cookie inspection

8. SECURITY RECOMMENDATIONS

8.1 Immediate Actions

- 1.  SQL Injection: Implement Prepared Statements
- 2.  CSRF: Implement Token Validation
- 3.  Security Headers: Add all missing headers
- 4.  Information Disclosure: Hide version details

8.2 Long-term Recommendations

- 1. **Update Software:**
 - Apache 2.4.58 → 2.4.66+
 - PHP 8.2.12 → 8.5.1+
 - OpenSSL 3.1.3 → 3.6.0+
- 2. **Implement WAF (Web Application Firewall)**
- 3. **Enable HTTPS in Production**
- 4. **Implement Strong Password Policy**
 - Minimum 12 characters
 - Special characters required
 - Regular password rotation
- 5. **Regular Security Audits**
 - Monthly vulnerability scans
 - Quarterly penetration testing

9. CONCLUSIONS

9.1 Achievements

1.  **Reconnaissance Complete:** Nmap, WhatWeb, Nikto scans
2.  **SQL Injection:** Found, exploited, and fixed
3.  **CSRF:** Found, exploited, and fixed
4.  **Configuration Hardening:** php.ini, httpd.conf updated
5.  **Documentation:** Complete report with screenshots

9.2 Key Learnings

- SQL Injection is still a critical threat
- CSRF attacks can be devastating without proper tokens
- Security headers are essential for modern web apps
- Configuration hardening is as important as code fixes
- Ethical hacking requires proper methodology